

High QPS Text Search Engine using Apache Solr and ZooKeeper

DBMS Term Project

Team Members:

23CS10012 - Charniya Krish
23CS10021 - Damiya Gondha
23CS30048 - Shatakshi Singh
23CS30049 - Shikhar Soni
23CS30050 - Shourya Godha

1 Objective

Modern applications require search systems that sustain high query volume while preserving low latency and stable relevance. This project designs and evaluates a scalable text search engine based on Apache Solr, with Apache ZooKeeper used for distributed coordination in SolrCloud mode.

The system supports full-text retrieval over a large corpus and is evaluated under increasing concurrency to study throughput limits, latency behavior, and service stability. The core objective is to understand how Queries Per Second (QPS) scales across standalone and distributed Solr deployments and to identify the practical trade-off between peak single-node performance and distributed fault tolerance.

2 Motivation and Problem Statement

Text search is foundational for e-commerce, documentation portals, digital libraries, and log analytics systems. The engineering challenge is not only accurate retrieval, but predictable performance under concurrent access.

2.1 QPS-Centric Performance Question

QPS is the primary throughput metric used in this report. As load increases, search systems generally move through three phases:

- **Linear scaling:** throughput rises with concurrency.
- **Saturation:** throughput plateaus as CPU, memory, or I/O limits are reached.

- **Degradation:** throughput drops and latency tail grows under resource contention.

This report investigates where those transitions occur across four Solr deployment configurations.

3 Methodology

3.1 High-Level Design

The architecture consists of a web-based client layer, Solr query/index nodes, and ZooKeeper as the coordination service for SolrCloud metadata, routing state, and failover support.



Figure 1: High-Level System Architecture

3.2 Standalone vs SolrCloud

Table 1: Comparison of Deployment Modes

Aspect	Standalone Solr	SolrCloud
Nodes	Single node	Multiple nodes
Sharding	Not used	Documents split across shards
Replication	Not used	Multiple replicas per shard
Fault tolerance	Limited	Automatic failover
Coordination service	Not required	ZooKeeper required
Query routing	Direct to node	Coordinated fan-out and merge

3.3 SolrCloud Query Processing Flow

1. Client sends query to an available Solr node.
2. Receiving node acts as coordinator.
3. Coordinator dispatches request to shard replicas.
4. Each shard performs local Lucene search.
5. Partial results are merged and re-ranked.
6. Final ranked response is returned to the client.

4 Technology Stack

Table 2: Technology Stack Used in the Project

Technology	Version	Purpose
Apache Solr	9.3.0	Full-text indexing and querying
Apache ZooKeeper	3.8.1	Cluster coordination for SolrCloud
Siege	Latest	HTTP workload generation
C++	C++17	Multithreaded benchmark client
Python	3.x	Data processing and result visualization
Java	11+	Solr runtime environment
CMake	3.14+	C++ build orchestration

5 Schema Design and Data Pipeline

5.1 Schema and Text Analysis Theory

The indexing strategy follows standard information retrieval principles:

- **Tokenization:** splits raw text into searchable terms.
- **Lowercasing:** improves matching consistency.
- **Stopword filtering:** removes low-information terms.
- **Query-time synonym expansion:** improves recall while avoiding index bloat.

5.2 Field Design Rationale

Table 3: Core Logical Fields and Their Role

Field	Type	Role in Retrieval
id	String	Unique document identifier
title	Analyzed text	High-signal searchable heading
content	Analyzed text	Main body text for relevance scoring
author	String	Exact filtering field
category/tags	Multi-value string	Faceted navigation
url/domain	String	Source metadata and filtering
last_modified	Date	Temporal metadata
text (copy field)	Analyzed text	Unified default search field

5.3 Data Pipeline

The ingestion flow is designed for stable large-batch indexing:

1. Extract and clean source corpus.
2. Transform records into Solr-compatible JSON documents.
3. Split dataset into chunks for controlled batch submission.
4. Index chunks into collection shards and verify availability.

6 Search Engine Behavior

6.1 Query Semantics

The retrieval flow supports full-text matching with relevance ranking, highlighting, faceting, and grouped presentation. Grouping by source file consolidates related hits and improves interpretability of multi-paragraph documents.

6.2 User-Facing Retrieval Features

- Full-text query processing over normalized fields.
- Snippet highlighting for matched terms.
- Faceted filtering by categorical metadata.

- Result grouping by source document.
- Pagination for stable browsing under large result sets.

7 Benchmarking Methodology

7.1 Workload Model

Two benchmark perspectives are used:

- HTTP-level stress benchmarking for throughput trends under concurrent users.
- Multithreaded client-side measurement for latency distribution and tail behavior.

7.2 Siege Benchmarking

Siege is used as a request-level load generator to evaluate end-to-end Solr query handling under controlled concurrency. The benchmark design focuses on throughput scalability by increasing concurrent clients and observing response stability over fixed-duration runs.

The Siege methodology in this project is based on the following principles:

- **Concurrency sweep:** multiple concurrency levels are tested to identify linear scaling, saturation, and collapse regions.
- **Continuous request pressure:** requests are sent without inter-request delays to approximate peak sustained demand.
- **Short controlled windows:** each run uses a fixed test window so measurements remain comparable across configurations.
- **HTTP-level observability:** transactions, availability, elapsed time, and transaction rate provide direct service-level throughput signals.

7.3 C++ Multithreaded Client Benchmarking

The C++ client complements Siege by exposing system behavior that average HTTP metrics can hide, especially latency tail characteristics. It models a high-throughput client with explicit thread management, connection reuse, and thread-safe metric aggregation.

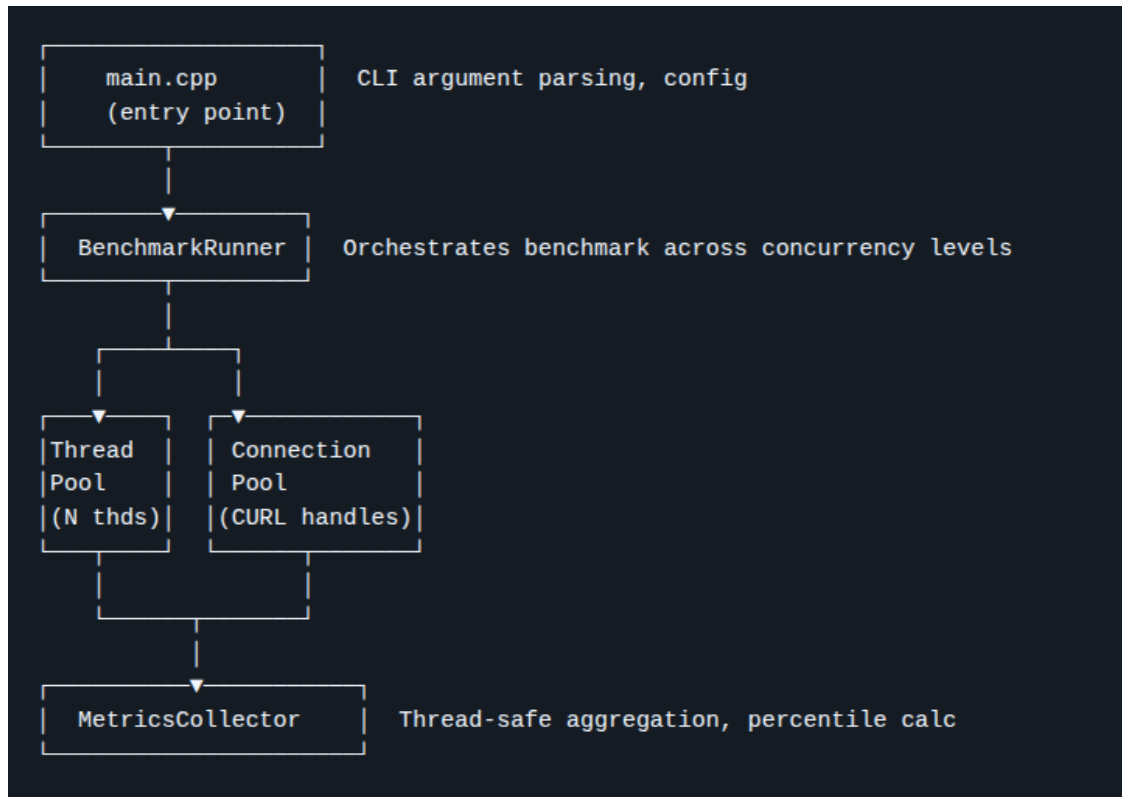


Figure 2: C++ Multithreaded Client Benchmarking System Architecture

Its theoretical value in the evaluation pipeline is:

- **Controlled parallelism:** worker-thread scheduling isolates the effect of client concurrency on server response behavior.
- **Connection reuse effects:** pooled HTTP connections reduce handshake overhead and better represent sustained production traffic.
- **Distribution-aware latency:** p50, p95, and p99 percentiles reveal median and tail response quality under load.
- **Queueing interpretation:** measured throughput and latency can be analyzed with Little’s Law to estimate effective in-system concurrency.

Together, Siege and the C++ benchmark provide complementary evidence: Siege gives broad HTTP throughput trends, while the C++ client provides deeper latency-structure and contention insights.

7.4 Primary Metrics

- **QPS:** successful queries per second.
- **Availability:** ratio of successful responses.
- **Latency percentiles:** p50, p95, p99 for median and tail response.

Throughput is expressed as:

$$QPS = \frac{SuccessfulQueries}{ElapsedTime(seconds)}$$

Effective system concurrency is interpreted with Little’s Law:

$$L = \lambda W$$

where L is average concurrent requests in-system, λ is throughput, and W is average latency.

8 Results/Screenshots

8.1 Benchmark Configurations

Table 4: Tested Solr Configurations

Config	Description	Nodes	Shards	Replicas	ZooKeeper
1	Standalone	1	—	—	—
2	SolrCloud (Small)	1	2	2	1
3	SolrCloud (Large)	3	3	4	1
4	SolrCloud (Full HA)	3	3	4	3

8.2 Configuration 1: Standalone Solr

Table 5: Standalone Performance Snapshot

Concurrency	Requests	Elapsed (s)	QPS
1.95	6970	9.09	766.78
2.93	17162	9.93	1728.30
4.86	18041	9.93	1816.82
6.74	36905	9.93	3716.52
10.64	46845	9.95	4708.04
12.63	41654	9.95	4186.33
16.61	39735	9.94	3997.48
22.62	37421	9.94	3764.69
40.58	38513	9.94	3874.55
58.46	34965	9.93	3521.15
77.94	32410	9.93	3263.85

8.3 Configuration 2: Single-Node SolrCloud

Table 6: 1-Node SolrCloud Performance Snapshot

Concurrency	Requests	Elapsed (s)	QPS
1.94	4989	9.71	513.80
2.90	13308	9.96	1336.14
0.76	2586	9.96	259.64

8.4 Configuration 3: Three-Node SolrCloud with One ZooKeeper

Table 7: 3-Node/1-ZK SolrCloud Performance Snapshot

Concurrency	Requests	Elapsed (s)	QPS
1.97	3371	9.43	357.48
2.92	6458	9.95	649.05
4.83	20543	9.95	2064.62
6.75	29345	9.96	2946.29
5.58	23721	9.96	2381.63
0.58	1468	9.96	147.39

8.5 Configuration 4: Three-Node SolrCloud with Three ZooKeeper Nodes

Table 8: 3-Node/3-ZK SolrCloud Performance Snapshot

Concurrency	Requests	Elapsed (s)	QPS
1.96	3230	9.61	336.11
2.94	7597	9.95	763.52
2.45	7955	9.94	800.30
1.30	3930	9.94	395.37
2.03	6682	9.94	672.23
1.76	4772	9.94	480.08

8.6 Peak Throughput Comparison

Table 9: Peak QPS by Configuration

Rank	Configuration	Peak QPS
1	Standalone	4708.04
2	SolrCloud (3 Nodes, 1 ZK)	2946.29
3	SolrCloud (1 Node, 1 ZK)	1336.14
4	SolrCloud (3 Nodes, 3 ZK)	800.30

8.7 Interpretation

- Standalone Solr delivered the highest single-host throughput.
- SolrCloud provided architecture-level scalability and failover semantics, but introduced overhead on shared hardware.
- Increasing ZooKeeper quorum size on the same host reduced effective query throughput due to additional coordination traffic.
- Beyond peak points, tail latency and instability increased sharply, consistent with resource contention and queueing effects.

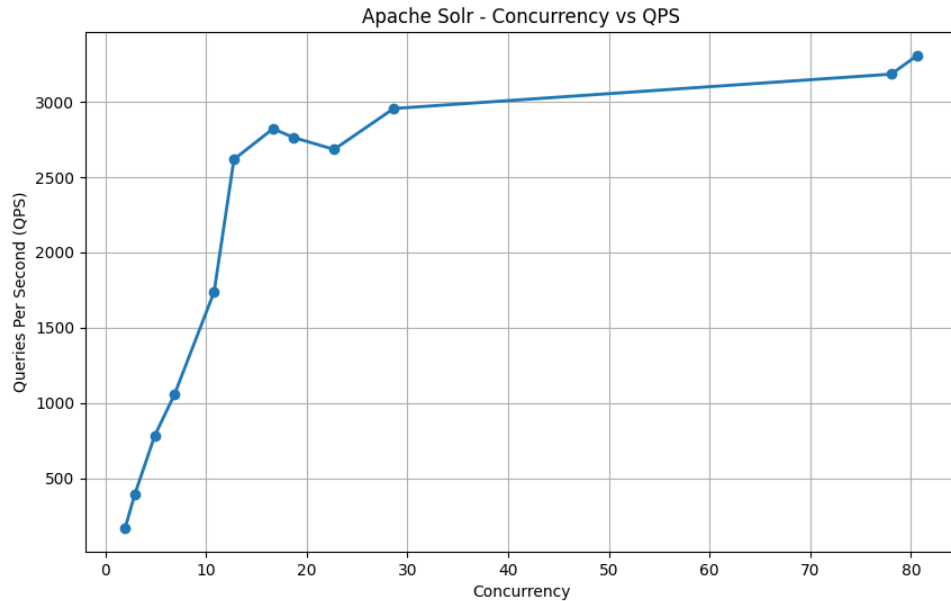


Figure 3: QPS trend across concurrency levels

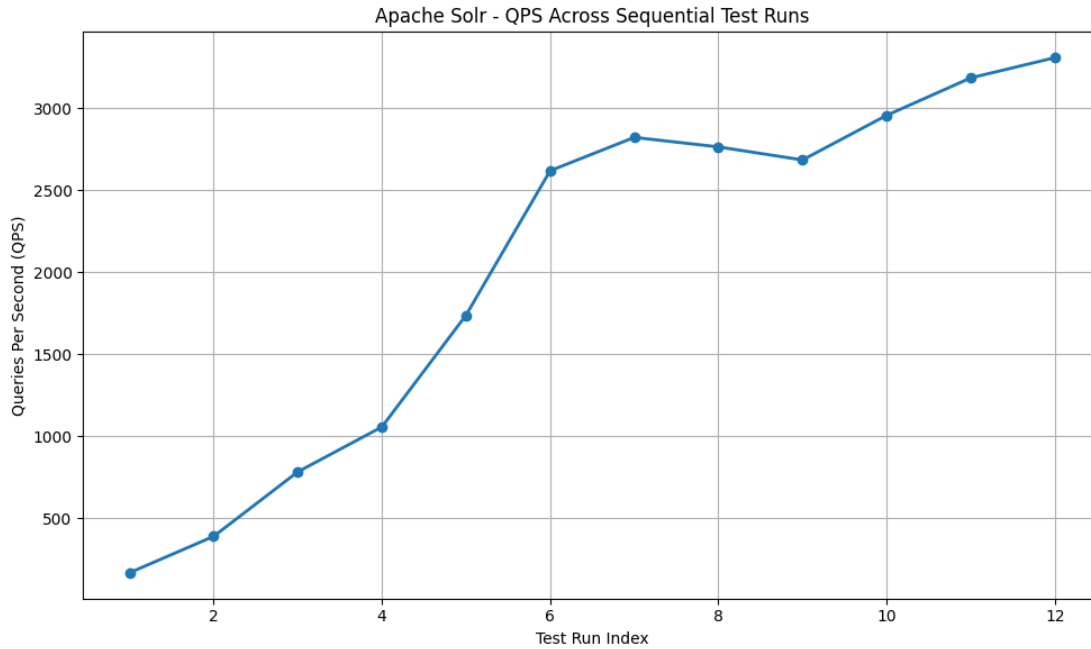


Figure 4: QPS trend across sequential test runs

8.8 Web Interface and Dashboard

The project includes a search interface and a benchmark dashboard for operational visibility. The dashboard summarizes peak throughput, comparative charts, and tabular benchmark outputs to support interpretation and reporting.

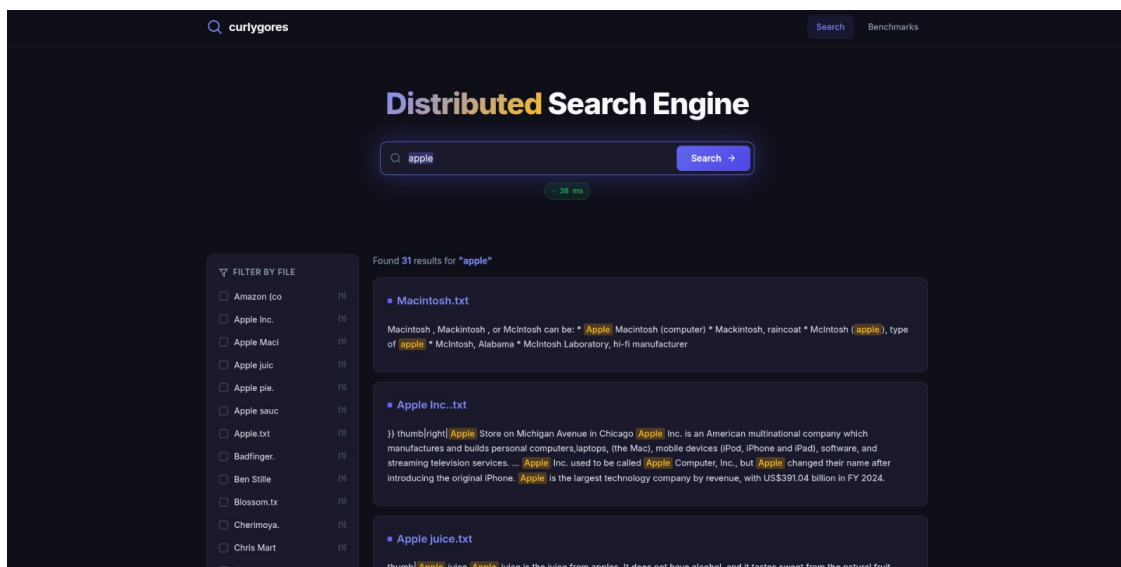


Figure 5: Search UI with query input, grouped results, and highlighted snippets

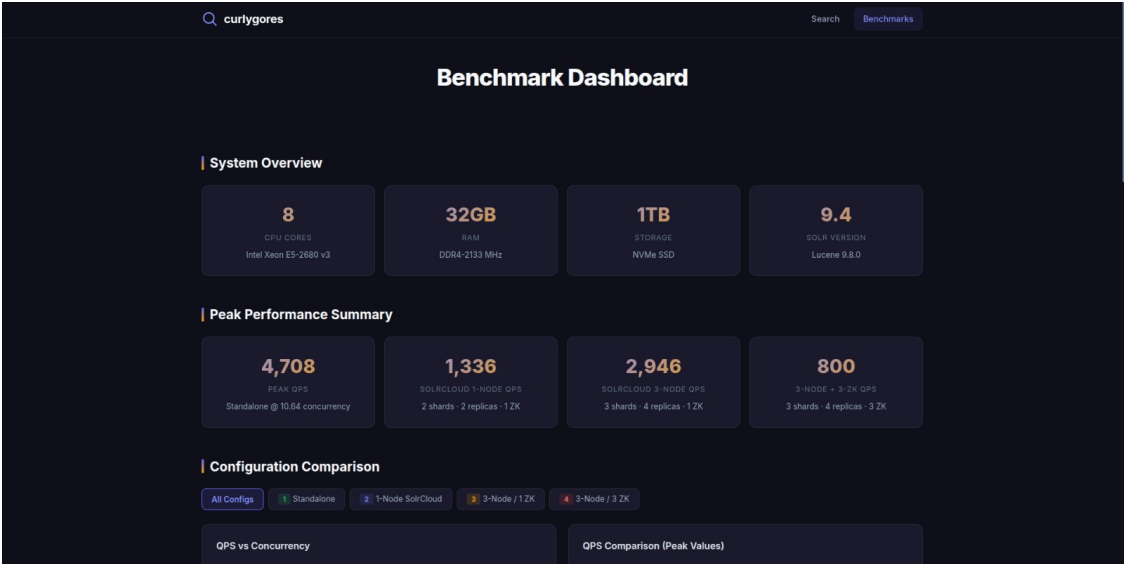


Figure 6: System Overview

The Raw Benchmark Data table provides a detailed view of the performance metrics for each configuration. The table is organized into four columns: Concurrency, Requests, Elapsed (s), and QPS. The data is presented for five configurations: Standalone, 1-Node Cloud, 3-Node / 12K, and 3-Node / 3 2K. The QPS values are highlighted in green.

Concurrency	Requests	Elapsed (s)	QPS
1.95	6,970	9.09	766.78
2.93	17,102	9.93	1728.30
4.86	18,041	9.93	1816.82
6.74	36,905	9.93	3716.52
10.64	46,845	9.95	4708.04
12.63	41,654	9.95	4188.33
16.61	38,735	9.94	3897.48
18.64	36,789	9.94	3701.11
22.62	37,421	9.94	3764.69
28.62	37,750	9.94	3797.79
30.58	37,371	9.94	3759.86
36.62	37,597	9.94	3782.39
40.58	38,513	9.94	3874.55
42.60	37,896	9.94	3812.47
46.60	37,993	9.94	3822.23
52.55	37,336	9.95	3752.36
58.46	34,965	9.93	3521.15
60.51	32,272	9.95	3243.42
68.46	37,297	9.97	3733.43

Figure 7: Results Table

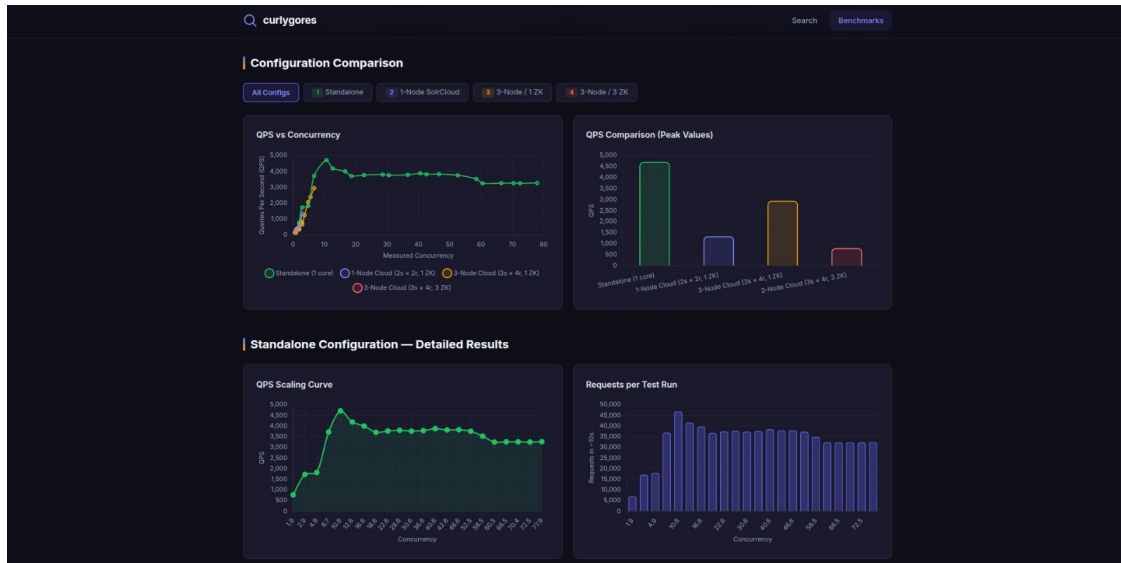


Figure 8: Analysis

9 Conclusions and Key Takeaways

1. Standalone Solr achieved the strongest single-machine throughput in this study.
2. SolrCloud is most valuable for distributed scalability and fault tolerance rather than maximum single-host QPS.
3. Coordination and network overhead are measurable and must be included in performance planning.
4. Throughput-only metrics are insufficient; percentile latency is essential for understanding user-visible quality under load.

10 References

1. Apache Solr Reference Guide - <https://solr.apache.org/guide/solr/latest/index.html>
2. Apache SolrCloud Documentation - <https://solr.apache.org/guide/solr/latest/deployment-guide/solrcloud.html>
3. Apache Lucene Documentation - <https://lucene.apache.org/core/documentation.html>
4. Apache ZooKeeper Documentation - <https://zookeeper.apache.org/documentation.html>
5. Siege Benchmarking Tool Manual - <https://www.joedog.org/siege-manual/>